C++ Memoization Guide

https://gemini.google.com/app/e7b2163c646b8aa8

User prompt: I have a recursive NumPaths function in C++ that recomputes the same subproblems many times, and I want to add memoization. Do not write the memoized version for me. Help me understand what exactly gets stored, when I check the table versus when I write to it, and how to tell an uncomputed cell from a legitimately computed zero. Then tell me what memoization costs me in space and whether that trade is always worth making.

Response: Memoization transforms a recursive function from an exponential tree of repeated work into a linear pass over unique states by caching the return value of each subproblem the first time it finishes.

1. What Exactly Gets Stored

A memoization table maps **function arguments (the state)** to **the function's return value**.

- **The Key (Indices):** Only the parameters that change between recursive calls and uniquely identify a subproblem. For a standard 2D grid path counter, this is your current coordinate pair `(r, c)`. Read-only data—like the grid dimensions, obstacle map, or target coordinates—do not affect the table dimensions.
- **The Value (Cell Content):** The exact integer returned by `NumPaths(r, c)`—meaning the total number of valid paths from `(r, c)` to the destination.
- **Data Type:** Path counts grow combinatorially (moving across a 20x20 grid yields over 137 billion paths), so table cells usually need a 64-bit integer (`long long` in C++) rather than a 32-bit `int`.

2. When to Check vs. When to Write

Think of the memo table as a gatekeeper at the top of your recursive function and a scribe at the bottom:

1. **Out-of-Bounds / Obstacle Check (First):** Before indexing into your memo table, check if `(r, c)` is outside the grid or on a blocked cell. Doing this first prevents out-of-bounds memory access on the table itself.
2. **Base Case Check:** If `(r, c)` is the destination, return `1` immediately (you can also store `1`, though returning directly is trivial).
3. **The Lookup (Before Recursing):** Check if `memo[r][c]` already holds a computed answer. If it does, **return that value immediately** and skip the recursive calls altogether.
4. **The Write (After Recursing, Before Returning):** If the cell is still marked as "uncomputed," make your recursive calls (e.g., `NumPaths(r + 1, c) + NumPaths(r, c + 1)`), store their sum into `memo[r][c]`, and return that stored value.

3. Distinguishing "Uncomputed" from a Computed `0`

In grids with obstacles or dead ends, a valid cell can legitimately have `0` paths to the exit. If you initialize your memo table with `0` and use `0` to mean "uncomputed," your function will think every dead-end cell has never been visited—and recompute the entire dead-end subtree every single time it reaches it, destroying your time complexity.

You have three idiomatic ways to solve this in C++:

Approach	How It Works	Trade-off
Sentinel Value (−1)	Initialize the table with −1 (e.g., `std::vector<std::vector<long long>>(R, std::vector<long long>(C, -1))`). Since path counts are never negative, −1 unambiguously means "uncomputed," while `0` means "computed: zero paths exist."	Zero extra memory overhead; requires signed integer types (`long long` instead of `uint64_t`).
Separate `visited` Table	Keep a parallel `bool visited[R][C]` (initialized to `false`) alongside your `memo` table. Check `visited[r][c]` before recursing, and set `visited[r][c] = true` when writing to `memo[r][c]`.	Works even when every possible integer is a valid return value; costs 1 extra byte per state.

Approach	How It Works	Trade-off
`std::optional<long long>`	Store `std::optional` in each cell. Check `.has_value()` on lookup and assign a value on write.	Very expressive and self-documenting, but doubles memory usage due to alignment padding.

Using **`-1` as a sentinel** is the standard choice for path-counting problems.

4. Space Cost and When the Trade-Off Is Worth It

Adding memoization to an $R \times C$ grid costs two things in memory:

- **The Memo Table:** $\mathcal{O}(R \times C)$ space (specifically, $R \times C \times 8$ bytes for `long long`).
- **The Call Stack:** $\mathcal{O}(R + C)$ space for the maximum recursion depth from start to finish. (Naive recursion *already* paid this call-stack cost.)

Is the trade-off always worth making? Almost always for runtime, as it collapses time complexity from $\mathcal{O}(2^{R+C})$ to $\mathcal{O}(R \times C)$, turning a computation that would take centuries on a 30x30 grid into microseconds. However, there are three cases where top-down memoization is **not** the optimal choice:

1. **You can use Bottom-Up DP with Space Optimization:** If you iterate row-by-row (tabulation) instead of recursing, computing row `r` only requires values from row `r - 1`. You can discard older rows and drop your space cost from $\mathcal{O}(R \times C)$ to $\mathcal{O}(\min(R, C))$ while eliminating recursion stack overhead entirely.
2. **Stack Overflow on Massive Grids:** If $R + C$ is on the order of 10^5 or more, top-down memoization can blow past the OS call stack limit even if the memo table fits in heap memory.
3. **No Obstacles Exist:** If the grid is completely open and only allows Down/Right moves, you don't need recursion or memoization at all—the answer is the binomial coefficient $\binom{(R-1)+(C-1)}{R-1}$, which can be computed in $\mathcal{O}(\min(R, C))$ time and $\mathcal{O}(1)$ space.

Summary: Store the path count indexed by `(r, c)`, check the table before recursing, write to it right before returning, and initialize with `-1` so computed `0`s aren't mistaken for unvisited states. It costs $\mathcal{O}(R \times C)$ memory, which is well worth the exponential speedup unless tight memory limits call for a space-optimized $\mathcal{O}(C)$ iterative approach.

---

*User prompt: below is my first attempt at achieving memoization: int RecurseNumPaths(int row, int col, int n, int** gridPaths); int NumPaths(int row, int col, int n) { int** gridPaths = new int*[n]; for (int i = 0; i < n; i++) { gridPaths[i] = new int[n](); } int result = RecurseNumPaths(row, col, n, gridPaths); // clean up memory for (int i = 0; i < n; i++) { delete[] gridPaths[i]; } delete[] gridPaths; return result; } int RecurseNumPaths(int row, int col, int n, int** gridPaths) { if (row == n || col == n) { return 1; } else { if (gridPaths[row][col] != 0) { return gridPaths[row][col]; } gridPaths[row][col] = RecurseNumPaths(row + 1, col, n, gridPaths) + RecurseNumPaths(row, col + 1, n, gridPaths); return gridPaths[row][col]; } }*

Response: Your core memoization structure—using a wrapper function (`NumPaths`) to allocate the table and a helper (`RecurseNumPaths`) to check before recursing and write before returning—is spot on! You also handled the `delete[]` cleanup properly and avoided copying the table by passing a pointer.

Here is a breakdown of what works, one subtle edge-case bug to watch out for, and how your choices connect to the concepts we just discussed:

1. Watch Out for the Off-by-One Boundary (`n` vs. `n - 1`)

Look closely at your array size versus your base case:

- You allocate `gridPaths` with indices from `0` to `n - 1`.

- Your base case checks `if (row == n || col == n) return 1;`.

For any recursive step generated *inside* the function, `row` and `col` stop at `n - 1` before indexing into `gridPaths`, and when they increment to `n`, the `if` statement catches them safely before touching `gridPaths[n]`.

- **The Risk:** If a caller ever invokes `NumPaths(n, 0, n)` directly, it returns `1` safely, *but* if your problem defines an $n \times n$ grid of cells (`0` to `n - 1`), the bottom-right corner is `(n - 1, n - 1)`. Reaching `row == n - 1 || col == n - 1` already forces a single straight line to the corner! By waiting until `row == n || col == n`, you are actually counting paths on an $(n+1) \times (n+1)$ grid of intersections (or an $n \times n$ grid of edges). Make sure that matches your problem statement—if `n` is the number of cells, your base case should be `row == n - 1 || col == n - 1`.

2. Why `0` Works Here (and When It Would Break)

You used `new int[n]()` with parentheses, which zero-initializes every cell to `0`, and then checked `if (gridPaths[row][col] != 0)`.

- **Why it works right now:** Because your grid has no obstacles and the base case returns `1`, every single cell is guaranteed to have at least `1` path to the end. Zero is impossible as a legitimate answer here, so `0` safely acts as your "uncomputed" sentinel!
- **When it would break:** The moment you add obstacles (e.g., `if (isBlocked(row, col)) return 0;`), some cells will legitimately evaluate to `0` paths. Note that `new int[n](-1)` is **not** valid C++ syntax for filling an array with `-1`—with raw arrays, you would either loop or use `std::fill_n(gridPaths[i], n, -1)`, or switch to `std::vector<std::vector<int>> gridPaths(n, std::vector<int>(n, -1));`.

3. Integer Overflow on Larger Grids

Because `gridPaths` and the return types are `int` (typically 32-bit signed, maxing out around 2.14×10^9), `NumPaths(0, 0, n)` will silently overflow into negative numbers starting around `n = 17` ($\binom{34}{17} = 2,333,606,220$). If you test this with larger `n`, swap `int` for `long long` for the return types and the table cells.

4. Modern C++ Memory Tip (`std::vector` vs. `new[]` / `delete[]`)

If your coursework or assignment requires raw `int**` pointers, your allocation and deallocation loop is completely right. If you are allowed to use the standard library, replacing the manual `new[]` and `delete[]` loops with a single `std::vector<std::vector<long long>>` (passed by reference `&` to `RecurseNumPaths`) eliminates 8 lines of boilerplate and guarantees no memory leaks if an exception is ever thrown.

Summary: Your memoization flow is completely functional and properly prunes the recursion tree in $\mathcal{O}(n^2)$ time. Just double-check whether your base case should trigger at `n - 1` (for an $n \times n$ grid of cells) or `n` (for an $n \times n$ grid of edges), and remember that using `0` as "uncomputed" only works here because there are no obstacles yet.

User prompt: Well in our problem, the lower left of the grid is represented as 1, 1, not 0, 0. so my base case is checking if we've reached the top most row or right most column if only up/right moves are allowed. after which only 1 path would be avail (so i return 1)

Response: That clarifies the geometry completely—and it also reveals a very sneaky **wasted memory slot** in how your 1-indexed coordinates interact with C++'s 0-indexed arrays!

Why Your Code Still Works (By a Lucky Coincidence)

Because your grid starts at `(1, 1)` and your base case triggers the moment `row == n` or `col == n`, look at which coordinates actually reach the memo table lookup:

- `row` and `col` start at `1` (or higher).
- Whenever either coordinate hits `n`, the `if (row == n || col == n)` check immediately returns `1` **before** touching `gridPaths`.

- That means `gridPaths[row][col]` is only ever indexed with values from `1` to `n - 1`!

Since `new int[n]` allocates valid indices from `0` to `n - 1`:

1. You **never go out of bounds** because index `n` is caught by the base case before the array access.
2. Row `0` and column `0` (`gridPaths[0][...]` and `gridPaths[...][0]`) are **allocated, zeroed out, and deleted, but never touched**, because your coordinates start at `1`.

Two Ways to Make the Indexing Intentional

Depending on how you want your code to read (or what an autograder / instructor might look for), you can align your table indices with your 1-based coordinates in one of two ways:

Approach	How to Adjust	Why Choose It
Option A: Shift the index (`row - 1`, `col - 1`)	Keep the allocation at `n` (or even shrink it to `n - 1` since row/col `n` hits the base case), and index using `gridPaths[row - 1][col - 1]`.	Zero wasted memory; if you ever changed the base case to `row == n && col == n` (storing the `1`s on the top/right borders inside the table), `gridPaths[n][...]` would crash with a segmentation fault unless you shift by `- 1`.
Option B: Allocate `n + 1` (`new int*[n + 1]`)	Allocate `n + 1` rows and `n + 1` columns, and keep indexing directly as `gridPaths[row][col]`.	Keeps 1-based problem math readable without `- 1` clutter everywhere, and makes index `n` valid memory if you ever inspect or store border values.

One Tiny Guardrail to Keep in Mind

What happens if a test case starts *on* the top row or rightmost column, e.g., `NumPaths(n, 1, n)`? Your code handles that cleanly and returns `1`. What if a degenerate test case passes a coordinate *outside* the grid (like `row > n`)? Changing `if (row == n || col == n)` to `if (row >= n || col >= n)` protects you from ever stepping past `n` into invalid memory.

Summary: Because your coordinates run from `1` to `n` and your base case catches `n` before indexing the array, `gridPaths` only uses indices `1` through `n - 1`—meaning it runs safely with zero out-of-bounds reads, while leaving row `0` and column `0` unused.

---

*User prompt: Ah this makes sense -- so reworked - the following code initializes a smaller grid to n - 1, while indexing with row - 1, col -1: int RecurseNumPaths(int row, int col, int n, int** gridPaths); int NumPaths(int row, int col, int n) { int** gridPaths = new int*[n - 1]; for (int i = 0; i < n; i++) { gridPaths[i] = new int[n - 1](); } int result = RecurseNumPaths(row, col, n, gridPaths); // clean up memory for (int i = 0; i < n; i++) { delete[] gridPaths[i]; } delete[] gridPaths; return result; } int RecurseNumPaths(int row, int col, int n, int** gridPaths) { if (row >= n || col >= n) { return 1; } else { if (gridPaths[row - 1][col - 1] != 0) { return gridPaths[row - 1][col - 1]; } gridPaths[row - 1][col - 1] = RecurseNumPaths(row + 1, col, n, gridPaths) + RecurseNumPaths(row, col + 1, n, gridPaths); return gridPaths[row - 1][col - 1]; } } MY SUMMARY: Originally, my approach lucked into utilizing the memo'd table, but was wasting memory allocation. now we initialize the smaller grid of possible paths and early returns at the top row/right column. By checking the table first, we still eliminate expensive recursion, and only write to it when full number of paths have been determined. And by passign the pointer to the table vs intializing a fresh table per call, we successfully use the memo.*

Response: Your summary captures the memoization lifecycle and pointer-passing mechanics accurately, and your shift to `[row - 1][col - 1]` inside `RecurseNumPaths` maps the 1-based coordinates to a 0-based `(n - 1) x (n - 1)` table with zero wasted slots.

However, shrinking `new int*[n - 1]` introduced **one critical out-of-bounds bug in your `for` loops** that will corrupt memory or crash at runtime:

1. The Loop Bound Bug (`i < n` vs. `i < n - 1`)

In `NumPaths`, you allocated the outer array with size `n - 1`:

```
int** gridPaths = new int*[n - 1]; // Valid indices: 0 to n - 2
```

Right below that, both your allocation loop and your `delete[]` loop still run up to `i < n`:

```
for (int i = 0; i < n; i++) { // BUG: When i == n - 1, gridPaths[n - 1] is out of bounds!
    gridPaths[i] = new int[n - 1]();
}
```

When `i` reaches `n - 1`, `gridPaths[n - 1]` writes past the end of the heap block you just allocated (undefined behavior / heap corruption), and `delete[] gridPaths[n - 1]` will likely trigger a segmentation fault.

The Fix: Update both `for` loop conditions in `NumPaths` from `i < n` to `i < n - 1`:

```
for (int i = 0; i < n - 1; i++) {
    gridPaths[i] = new int[n - 1]();
}
// ... and in cleanup:
for (int i = 0; i < n - 1; i++) {
    delete[] gridPaths[i];
}
```

2. Guarding Against `n <= 1`

What if a test case calls `NumPaths(1, 1, 1)` (a 1×1 grid)?

- With `n = 1`, `new int*[n - 1]` allocates `new int*[0]`. While C++ technically permits zero-length dynamic array allocations, you don't even need the table when `row >= n || col >= n` is immediately true at the start.
- Adding a quick early check at the very top of `NumPaths`—`if (row >= n || col >= n) return 1;`—saves you from allocating and deleting a table when the starting coordinate is already on (or past) the top/right boundary.

Summary: Your conceptual summary and `[row - 1][col - 1]` indexing logic are completely solid. Just change the two `for` loop conditions in `NumPaths` from `i < n` to `i < n - 1` so your allocation and cleanup don't write past the end of `new int*[n - 1]`.

User prompt: Ah; so based on this, I predict the memo call count at n=5 ~25 (because the run time is reduced to O(r - 1 x c - 1) vs. the non-memo version being 1024 calls O(2^(r -1 + c -1). the actual call counts were 33 for memoized and 139 for original -- which based on the runtime comps, the tree version is already beginning to outpace the calls of the memo'd version

Response: Your intuition about the growth curves is spot on, and there is a neat exact mathematical formula behind both **33** and **139** that explains why they differ from the raw Big-O expressions $(n-1)^2$ and $2^{2(n-1)}$.

Big-O hides constant factors and boundary effects. Here is where those exact call counts come from:

1. Why the Memoized Version Makes Exactly 33 Calls (at $n = 5$)

At $n = 5$, your memo table has $(5-1) \times (5-1) = \mathbf{16}$ interior cells (coordinates where both `row` and `col` are between 1 and 4).

Every one of those 16 interior cells is computed **exactly once**. And when an interior cell is computed for the first time, it makes **2 recursive calls** (one Up, one Right):

- **Initial call** to `RecurseNumPaths(1, 1, 5, gridPaths)`: 1 **call**
- 16 **unique interior cells**, each making 2 outgoing calls: $16 \times 2 = \mathbf{32}$ calls
- **Total calls:** $1 + 32 = \mathbf{33}$ calls

Wait—where did the extra 17 calls go if there are only 16 cells in the table?

- 8 **calls** hit the base case (`row == 5` or `col == 5`) and immediately returned 1.
- 9 **calls** hit an interior cell that was *already computed* (`gridPaths[row - 1][col - 1] != 0`) and immediately returned the cached value without spawning children!

In fact, for any $n \geq 1$, the exact number of calls made by your memoized function starting from `(1, 1)` is:

$$\text{Memoized Calls}(n) = 2(n-1)^2 + 1$$

Notice the leading coefficient of 2: in Big-O notation, $2(n-1)^2 + 1$ simplifies to $\mathcal{O}(n^2)$, which is why your ~ 25 estimate was right in the ballpark!

2. Why the Unmemoized Version Makes Exactly `139` Calls (Not `1024`)

Your $2^{2(n-1)}$ formula assumes a **full binary tree** of depth $2(n-1) = 8$, where every branch goes all the way down 8 levels before stopping.

However, your base case (`row == n || col == n`) cuts branches short the moment *either* coordinate hits 5! For example, moving Up 4 times in a row (`(1,1) -> (2,1) -> (3,1) -> (4,1) -> (5,1)`) hits the base case at depth 4, pruning the rest of that subtree.

Because every leaf in the recursion tree returns 1, and every internal node returns the sum of its two children, the number of **leaves** in the unmemoized recursion tree is equal to the total number of paths:

$$\text{Leaves} = \text{NumPaths}(1, 1, 5) = \binom{8}{4} = \mathbf{70}$$

In any strict binary tree (where every node has either 0 or 2 children), a classic graph theory property states that the total number of nodes is always $2 \times \text{Leaves} - 1$:

$$\text{Unmemoized Calls}(n) = 2\binom{2n-2}{n-1} - 1$$

Plug in $n = 5$:

$$2 \times 70 - 1 = \mathbf{139} \text{ calls}$$

3. Watching the Exponential Explosion Take Over

At $n = 5$, 139 vs. 33 is already a $4.2\times$ speedup. Because $\binom{2n-2}{n-1}$ grows asymptotically as $\mathcal{O}\left(\frac{4^n}{n}\right)$, watch how fast the gap widens as n increases:

Grid Size (n)	**Total Paths** $\binom{2n-2}{n-1}$	**Memoized Calls** $2(n-1)^2+1$	**Unmemoized Calls** $2\binom{2n-2}{n-1}-1$	**Speedup Factor**
$n = 3$	6	9	11	$1.2\times$
$n = 5$	70	33	139	$4.2\times$
$n = 8$	$3,432$	99	$6,863$	$69\times$
$n = 12$	$705,432$	243	$1,410,863$	$5,806\times$
$n = 16$	$155,117,520$	451	$310,235,039$	$687,882\times$

Grid Size (n)	Total Paths $\binom{2n-2}{n-1}$	Memoized Calls $2(n-1)^2+1$	Unmemoized Calls $2\binom{2n-2}{n-1}-1$	Speedup Factor
$n = 20$	$35,345,263,800$	723	$70,690,527,599$	$\sim 97.7 \text{ million}\times$

By $n = 20$, your memoized version finishes in 723 **function calls** (microseconds), while the original recursive version makes over 70.6 billion **calls** (taking minutes to hours).

Summary: The memoized version makes $2(n-1)^2+1 = 33$ calls because each of the 16 table cells spawns 2 lookups plus the 1 root call, while the unmemoized version makes $2\binom{2n-2}{n-1}-1 = 139$ calls because branches hitting `row == n` or `col == n` terminate early rather=======than growing to a full depth-8 binary tree.

---

*User prompt: wait - my memo check *is* before the recursive call..*

Response: You are completely right—that was sloppy phrasing on my part! Your memo check (`if (gridPaths[row - 1][col - 1] != 0)`) is indeed right at the top of `RecurseNumPaths`, **before** the function makes its two recursive calls. And that is the cleanest, most standard place to put it.

What I was poorly hinting at is the difference between **checking Inside the Child (at entry)**—which is what your code does—versus **checking in the Parent (before calling the child)**:

1. What Your Code Does: Check at Function Entry ("Call, Then Check")

When `(1, 1)` finishes computing its Up neighbor `(2, 1)` and moves on to its Right neighbor `(1, 2)`, many cells inside `(1, 2)`'s subtree—like `(2, 2)`—were **already computed** during the `(2, 1)` branch.

In your code, when `(1, 2)` wants the answer for `(2, 2)`:

1. `(1, 2)` still invokes `RecurseNumPaths(2, 2, n, gridPaths)` (**this increments the call counter by 1**).
2. Inside that call, `(2, 2)` immediately hits `if (gridPaths[1][1] != 0)` and returns the cached answer without recursing further.

That is where those 9 **cache-hit calls** and 8 **base-case calls** came from in the 33-call total (16 first-time visits + 9 cache-hit visits + 8 base-case visits = 33 calls).

2. The Alternative: Check in the Parent ("Check Before Calling")

If someone wanted the absolute minimum number of function invocations, the parent could peek at `gridPaths[row][col - 1]` *before* even invoking `RecurseNumPaths(row + 1, col, ...)` and only call the function if the cell is `0`.

- Doing that drops the call count at $n = 5$ from 33 down to 16 (exactly 1 call per interior cell).
- **Why nobody writes it that way:** It makes the code messy and repetitive because you have to duplicate both the boundary checks (`row + 1 >= n`) and the table lookups twice inside the parent before every recursive call.

Summary: Your memo check is already in the textbook-proper spot at the top of `RecurseNumPaths`. Because each of the 16 interior cells calls `RecurseNumPaths` on both of its neighbors and lets the child function check the base case or memo table upon entry, you get $1 + 2(16) = 33$ total calls.

User prompt: What would peeking at gridPaths[row][col - 1] == RecurseNumPaths(row + 1, col, ...) wouldn't that correspond to RecurseNumPaths(row, col + 1, ...)?

Response: Look at the index math when you apply your `- 1` shift to both coordinates of `(row + 1, col)`:

- **New Row:** `(row + 1) - 1` simplifies to **`row`**
- **New Col:** `(col) - 1` stays **`col - 1`**

So `gridPaths[row][col - 1]` actually *is* the table slot for `(row + 1, col)`!

Here is the side-by-side mapping of all three states in 1-based coordinates versus their 0-based table indices:

State (1-based)	Coordinate Formula	Shifted Table Index (- 1 on both)	Simplified Table Index
Current Cell	`(row, col)`	`gridPaths[row - 1][col - 1]`	`gridPaths[row - 1][col - 1]`
Up Neighbor	`(row + 1, col)`	`gridPaths[(row + 1) - 1][col - 1]`	**`gridPaths[row][col - 1]`**
Right Neighbor	`(row, col + 1)`	`gridPaths[row - 1][(col + 1) - 1]`	**`gridPaths[row - 1][col]`**

Your question actually proves the exact point of why nobody checks the table in the parent! Because the `- 1` index shift cancels out the `+ 1` step on the coordinate you are moving along, `gridPaths[row][col - 1]` *looks* like you forgot to increment `row` and decremented `col` instead.

By keeping the check at the top of `RecurseNumPaths` like you did, every call only ever looks at `gridPaths[row - 1][col - 1]`, and you never have to mentally juggle `(row + 1) - 1` vs. `col - 1`.

Summary: Because of the `- 1` shift for 0-based indexing, moving to `(row + 1, col)` translates to `gridPaths[(row + 1) - 1][col - 1]`, which simplifies to `gridPaths[row][col - 1]`—whereas `(row, col + 1)` simplifies to `gridPaths[row - 1][col]`.

User prompt: hm. now there's a creative feature i'd like to add. the space tradeoff: my N×N table vs. recursion depth seems worth it at high enough N. but maybe the trade isn't "always worth it". for tiny n, memo can be slower than the plain version. at n=1–3, time is under ms, but at n=14, non-memo'd version takes forever

Response: You just rediscovered one of the classic techniques in high-performance library design: **hybrid dispatch** (or **threshold switching**).

Real-world standard libraries do this all the time. For example, `std::sort` (Introsort) uses Quicksort for large arrays, but switches to Insertion Sort once a partition drops below ~16 elements because the constant-factor overhead of Quicksort outweighs its $\mathcal{O}(N \log N)$ advantage at tiny scales.

Here is why your observation holds true in C++ and how you can turn it into a smart feature.

1. Why Plain Recursion Beats Memoization at Tiny `n`

At `n = 3`:

- **Unmemoized (11 calls):** Every call just pushes a few registers onto the **CPU stack** (which is already hot in L1 cache) and does simple integer addition. Total time: a few dozen nanoseconds.
- **Memoized (9 calls):** Before doing any math, `NumPaths` makes n **separate calls to the OS heap allocator** (1 for `new int*[n - 1]` and `n - 1` for each `new int[n - 1]()`), zero-initializes the memory, chases double pointers (`gridPaths[r][c]`), and then makes n **calls to `delete[]`**. Heap allocation (`new`/`malloc`) typically costs hundreds of CPU cycles per call—meaning at `n = 2` or `3`, you spend 10x more time asking the OS for memory than actually counting paths!

2. Three Ways to Build Your Hybrid Feature

Depending on how sophisticated you want to get, you can implement this threshold in three ways:

Strategy	How It Works	Best For
1. Top-Level Entry Switch	At the very top of `NumPaths(row, col, n)`, check the remaining distance to the corner: `int rem = (n - row) + (n - col);`. If `rem <= THRESHOLD` (e.g., `n <= 4`), immediately call a plain, allocation-free `NaiveNumPaths(row, col, n)` and skip `new[]` / `delete[]` completely.	Eliminating heap allocation overhead altogether on small inputs.
2. Subtree Cutoff (Inside Recursion)	Even when `n = 15`, as the recursion approaches the top-right corner `(n, n)`, the remaining subgrid becomes tiny (e.g., a 3×3 corner left to go). If `(n - row) <= 2 && (n - col) <= 2`, switch to naive recursion for the last few steps instead of looking up the table.	Avoiding pointer-chasing near the boundary (though in practice, since the table is already allocated, checking the cache is still better if multiple paths converge there).
3. Stack Buffer Optimization (Small-Buffer Optimization)	Instead of switching algorithms, switch *where the memory lives*: for small `n` (say `n <= 16`), use a fixed-size stack array `int stackGrid[16][16] = {0};` (0ns allocation cost!), and only use `new[]` on the heap when `n > 16`.	Getting both $\mathcal{O}(n^2)$ call counts *and* zero heap-allocation overhead.

3. Finding Your Exact Crossover Point (`n_threshold`)

Notice that in **Strategy 1**, you want to base the decision on the **remaining grid dimensions**—specifically `n - row` and `n - col`—not just `n`!

- Why? Because if someone calls `NumPaths(13, 13, 15)` on a 15×15 grid, they are already 2 steps away from the corner (`(13, 13)` to `(15, 15)` is effectively an `n = 3` problem!). Allocating a 14×14 heap table just to walk a 3×3 corner would waste huge amounts of time and space.

If you check `if (n - row <= 3 || n - col <= 3)` at the very start of `NumPaths`:

- Any small grid (`n <= 4`) bypasses heap allocation entirely.
- Any query starting near the top or right edge of a massive grid *also* bypasses heap allocation.

Summary: Heap allocation (`new[]` and `delete[]`) has a high fixed cost that outweighs memoization's call savings on tiny grids (`n <= 4`). Adding a fast-path check at the start of `NumPaths` based on the remaining distance (`n - row` and `n - col`) lets you dispatch small subproblems to an allocation-free recursive helper while reserving the heap table for large grids.

User prompt: not yet. My approach uses zeros to represent non-computed cells. this is due to fact that any non-base cell in my grid can't legitimately be 0. What changes would I need to take if cells could be blocked? defend sentinel advice for both case where i have blocked cells vs unblocked cells

Response: If cells can be blocked, **4 specific parts** of your current design must change—including a subtle trap in your base case that most people miss at first glance.

Part 1: The 4 Changes Needed When Cells Can Be Blocked

1. Initialization & Lookup Sentinel (`0` $\rightarrow$ `-1`)

Because a cell trapped behind walls legitimately has `0` paths to `(n, n)`, `0` can no longer mean "uncomputed."

- **Change:** You can no longer rely on `new int[n - 1]()` to zero-initialize the table. You must explicitly fill every cell with `-1` (e.g., using `std::fill_n(gridPaths[i], n - 1, -1);` right after allocating each row), and change your lookup check from `!= 0` to `!= -1` (or `>= 0`).

2. A New Blocked-Cell Base Case (Checked *Before* the Boundary!)

At the very top of `RecurseNumPaths`, you must check whether the current `(row, col)` is blocked:

- If `isBlocked(row, col)` is true, immediately **`return 0;`**.
- Notice that this check must happen **first**—even before your boundary base case or memo lookup—in case the starting cell itself or a boundary cell is blocked.

3. The Subtle Trap: Your Top/Right Boundary Base Case Must Change!

Look at your current shortcut base case:

```
if (row == n || col == n) return 1;
```

Why that breaks with obstacles: In an unblocked grid, reaching the top row (`row == n`) or rightmost column (`col == n`) guarantees `1` path because you can just slide straight along the wall to `(n, n)`. **With obstacles, a cell along the top wall or right wall might be blocked!** If `(n, 3)` is blocked, then `(n, 1)` actually has **`0` paths** to `(n, n)`, not `1`.

- **Change:** You can no longer stop early at `row == n || col == n`.
  1. **Out-of-bounds check:** `if (row > n || col > n) return 0;` (stepping off the grid yields `0` paths).
  2. **Obstacle check:** `if (isBlocked(row, col)) return 0;`
  3. **True destination base case:** `if (row == n && col == n) return 1;` (only return `1` when you actually land on the top-right corner).

4. Table Size Must Expand from `n - 1` Back to `n`

Because cells on row `n` and column `n` are no longer automatic base cases, `RecurseNumPaths` will now recurse along row `n` and column `n`—and needs to memoize them! Therefore, your 0-indexed table must be allocated as `n x n` (`new int*[n]`), indexed by `[row - 1][col - 1]`.

Part 2: Defending the Sentinel Choice in Both Scenarios

Here is why **`0` is the right engineering choice for your unblocked grid**, while **`-1` is mandatory for the blocked grid**:

Scenario	Recommended Sentinel	Engineering Defense
Case A: Unblocked Grid *(Your current problem)*	**`0` is optimal** (though `-1` is still good practice)	**1. Mathematical Guarantee:** Because every cell (r, c) has at least one monotonic path to (n, n), the codomain of `NumPaths` is strictly ≥ 1. Therefore, 0 lies completely outside the set of valid return values, making it a mathematically collision-free sentinel. **2. Hardware & OS Synergy:** In C++, `new int[n - 1]()` value-initializes memory to `0`. Under the hood, the compiler translates this to `calloc` or `memset(..., 0, ...)`, which is heavily optimized at the CPU/OS level (and for large allocations, the OS can hand out pre-zeroed virtual memory pages at nearly zero cost). Using -1 requires an explicit loop writing `0xFFFFFFFF` to every integer. **3. Unsigned Range:** Using `0` lets you use `uint64_t` (`unsigned long long`), doubling your positive integer capacity before overflow compared to signed `long long`.
Case B: Blocked Grid *(With obstacles)*	**`-1` is mandatory**	**1. Prevents Exponential Collapse:** With obstacles, the codomain of `NumPaths` expands to ≥ 0. If you keep `0` as your sentinel, any cell that cannot reach (n, n) computes `0 + 0 = 0` and writes `0` to the table. The next time another path

Scenario	Recommended Sentinel	Engineering Defense
		reaches that dead-end cell, `gridPaths[r][c] != 0` evaluates to `false`! The function thinks the dead end was never visited and **recomputes the entire dead-end subtree from scratch**, degrading worst-case runtime right back to exponential $\mathcal{O}\left(2^{2n}\right)$. **2. Zero Extra Space Overhead:** Using -1 separates the state "uncomputed" (-1) from "computed zero paths" (`0`) inside the exact same integer without needing a second `bool visited[n][n]` array.

Why many engineers still default to `-1` even in unblocked DP: In production or interview settings, requirements frequently evolve mid-problem (just like "now add obstacles!"). Starting with -1 makes your memoization logic **invariant to whether `0` is a valid answer**, preventing the silent performance bug where a code change makes `0` achievable without crashing or throwing a compiler warning.

Summary: Adding blocked cells requires switching your sentinel from `0` to -1, checking for obstacles first, expanding the table to `n x n`, and changing the base case from `row == n || col == n` to `row == n && col == n` (with out-of-bounds returning `0`) because walls along the top or right edges can block the straight-line path to the corner. For unblocked grids, `0` is faster to initialize and mathematically safe since paths are always ≥ 1, whereas for blocked grids, -1 is essential so dead-end cells returning `0` aren't endlessly recomputed.

---

*User prompt: no - for creative feature - i considered the instrumentation for counting calls as I've kinda already implemented as follows: int recurseNumCalls = 0; int RecurseNumPaths(int row, int col, int n, int** gridPaths) { recurseNumCalls++; if (row >= n || col >= n) { return 1; } else { if (gridPaths[row - 1][col - 1] != 0) { return gridPaths[row - 1][col - 1]; } gridPaths[row - 1][col - 1] = RecurseNumPaths(row + 1, col, n, gridPaths) + RecurseNumPaths(row, col + 1, n, gridPaths); return gridPaths[row - 1][col - 1]; } } I do something similar for iterative numpaths. void printNumCalls(int& calls) { cout << "Num calls: " << calls << endl; cout << "Resetting.." << endl; calls = 0; } then i can call that, passing in the call counters, and zero out for subsequent calls*

Response: Your instrumentation setup is a clean, hardware-independent way to prove the algorithmic speedup. Relying on wall-clock timers (`std::chrono`) for tiny inputs like $n = 3$ is notoriously noisy because OS background tasks or CPU frequency scaling can skew microsecond measurements, whereas **call counting is 100% deterministic**. Passing `int& calls` by reference into `printNumCalls` so it prints and resets in one step is also a great touch—it prevents forgotten resets from polluting the next test case.

To make this instrumentation feature really shine (especially if this is for a report, lab demo, or assignment), you can level it up in three easy ways:

1. Break `recurseNumCalls` into 3 Sub-Metrics (The "Cache Profiler")

Right now, `recurseNumCalls++` at the top of `RecurseNumPaths` gives you the **Total Calls** (33 at $n = 5$). By adding two more counters inside your existing `if` branches, you can show *exact proof* of how memoization prunes the tree:

- **`baseCaseHits++`**: Inside `if (row >= n || col >= n)`
- **`memoHits++`**: Inside `if (gridPaths[row - 1][col - 1] != 0)` — **every increment here represents an entire recursive subtree that your memo table just saved you from computing!**
- **`memoMisses++`**: Right before `gridPaths[row - 1][col - 1] = ...` (this will always equal $(n-1)^2$, the exact number of unique subproblems).

Notice how at $n = 5$, printing that breakdown shows:

```
Total Calls: 33 | Base Cases: 8 | Cache Hits (Subtrees Pruned): 9 | Cache Misses (Computed): 16
```

And notice theinvariant: `Base Cases + Cache Hits + Cache Misses == Total Calls` ($8 + 9 + 16 = 33$).

2. Bundle the Counters in a `struct` Instead of Loose Globals

If you have counters for both the naive and memoized versions, wrapping them in a small struct makes `printNumCalls` even cleaner and lets you compute the **Speedup Ratio** and **Calls Saved** automatically:

```
struct CallStats {
    long long totalCalls = 0;
    long long memoHits = 0;
    long long baseCases = 0;

    void printAndReset(const string& label) {
        cout << "[" << label << "] Total Calls: " << totalCalls
             << " (Base: " << baseCases << ", Cache Hits: " << memoHits << ")\n";
        totalCalls = memoHits = baseCases = 0;
    }
};
```

(Tip: Use `long long` for the unmemoized call counter if you test $n \geq 17$, because the call count is $2\times$ the number of paths and will overflow a 32-bit `int` even faster than the path count itself!)

3. Add a Side-by-Side Comparison Loop (`n = 1` to `15`)

Since your `printNumCalls` resets the counters after each run, you can run a single `for (int n = 1; n <= 15; n++)` loop that calls both your unmemoized and memoized functions, prints their call counts side-by-side, and computes `unmemoizedCalls / memoizedCalls` (or `% of calls pruned: (1.0 - (double)memoCalls / naiveCalls) * 100`). Watching the "Calls Pruned" column climb from `0%` at $n = 2$ to `99.9998%` at $n = 15$ tells the whole story of your optimization at a glance.

Summary: Your `recurseNumCalls` counter and pass-by-reference `printNumCalls(int& calls)` reset helper provide a deterministic benchmark free from OS timing noise. Adding a counter inside the `gridPaths[row - 1][col - 1] != 0` branch to track **cache hits** (and using `long long` for large n) lets you report not just total calls, but the exact number of redundant subtrees pruned by your table.

User prompt: Goiing with the 3 Sub-metrics (cache-profiler)

Response: Here is how the three sub-metrics slot directly into your existing `RecurseNumPaths` branches, along with an updated `printNumCalls` helper that verifies your total and resets all four counters at once:

```
long long recurseNumCalls = 0;
long long baseCaseHits = 0;
long long memoHits = 0;
long long memoMisses = 0;

int RecurseNumPaths(int row, int col, int n, int** gridPaths) {
    recurseNumCalls++;

    if (row >= n || col >= n) {
        baseCaseHits++; // 1. Hit the top or right boundary
        return 1;
    } else {
        if (gridPaths[row - 1][col - 1] != 0) {
            memoHits++; // 2. Found in table (pruned an entire subtree!)
            return gridPaths[row - 1][col - 1];
        }

        memoMisses++; // 3. First time visiting this interior cell
        gridPaths[row - 1][col - 1] =
            RecurseNumPaths(row + 1, col, n, gridPaths) +
```

```
            RecurseNumPaths(row, col + 1, n, gridPaths);
        return gridPaths[row - 1][col - 1];
    }
}

void printMemoStats(long long& total, long long& base, long long& hits, long long& misses) {
    cout << "Total Calls:  " << total << "\n"
         << "  ├─ Base Cases:   " << base << "\n"
         << "  ├─ Cache Hits:   " << hits << " (subtrees pruned)\n"
         << "  └─ Cache Misses: " << misses << " (unique cells computed)\n";

    if (hits + misses > 0) {
        double hitRate = (100.0 * hits) / (hits + misses);
        cout << "Interior Cache Hit Rate: " << hitRate << "%\n";
    }
    cout << "Resetting counters...\n\n";

    total = base = hits = misses = 0;
}
```

Three Cool Patterns Your Profiler Will Reveal

When you run this across different grid sizes starting from `(1, 1)`, every single counter follows an exact mathematical law that makes for great talking points:

1. **Built-In Sanity Check:** Because the three branches are mutually exclusive and cover every execution path, `baseCaseHits + memoHits + memoMisses` **must always equal** `recurseNumCalls`.
2. **Predictable Subproblem Count:** `memoMisses` will always equal $(n-1)^2$—proving that your memoized function never computes the same interior cell twice.
3. **Predictable Pruning:**
   - `baseCaseHits` always equals $2(n-1)$ (one entry into row `n` for each of the $n-1$ columns, and one entry into col `n` for each of the $n-1$ rows).
   - `memoHits` always equals $(n-2)^2$ (every interior cell except Row 1 and Col 1 gets reached a second time from its other neighbor and hits the cache!).

Grid Size (n)	**`recurseNumCalls`** $2(n-1)^2+1$	**`baseCaseHits`** $2(n-1)$	**`memoHits`** $(n-2)^2$	**`memoMisses`** $(n-1)^2$	**Interior Cache Hit Rate** $\frac{\text{hits}}{\text{hits+misses}}$
$n=2$	3	2	0	1	0.0%
$n=3$	9	4	1	4	20.0%
$n=5$	33	8	9	16	36.0%
$n=10$	163	18	64	81	44.1%
$n=15$	393	28	169	196	46.3%

*(Notice why the Interior Cache Hit Rate approaches 50% as n grows: each interior cell is visited at most **twice**—once from below as a **Cache Miss**, and once from the left as a **Cache Hit**!)*

Summary: Placing counters inside each of the three branches of `RecurseNumPaths` decomposes your total calls into boundary exits ($2(n-1)$), pruned subtrees ($(n-2)^2$), and unique table writes ($(n-1)^2$), giving you a self-verifying breakdown where the three sub-metrics always sum to `recurseNumCalls`.
